

Digital payments revisited

April 24, 2026

An attempt at an Ideal Functionality for a Payment System

- Formalise an **ideal functionality** $\mathcal{F}_{\text{pay}}$
- Capture:
 - Security (no fraud, correctness)
 - Fault-tolerance (network uncertainty)
 - Privacy (minimal leakage)
- Based on:
 - Trust: banks (only for account holder's privacy and accounting) and regulator trusted. Banks should not learn the purpose of the expenditure or the payee's identity. All other parties untrusted.
 - Users potentially malicious
 - Network unreliable but eventually delivers

- Payer P_0 , Payee P_1
- Banks B_0, B_1
- Regulator R
- Adversary $\mathcal{A}$ (controls scheduling of messages)

Design principle:

- No trusted intermediaries inside functionality

Per user U :

- Bank $B(U)$
- Account $\text{acc}(U)$
- Authentication $\text{auth}(U)$
- Identifier $\text{uid}(U)$

Balances:

$\text{bal}[\text{acc}(U)]$

Per transaction tx :

- $\text{tid}, (P_0, P_1, m)$
- Status $\in \{\text{pending}, \text{committed}, \text{aborted}\}$
- Evidence log $E(tx)$

Interface: Registration

$(\text{Register}, U, B(U), \text{acc}(U), \text{auth}(U), \text{uid}(U))$

- Store user mapping
- Bind identity to bank and account

Abstraction:

- Captures setup without modelling implementation details

Interface: Payment Request

$(\text{PayRequest}, P_0, P_1, m, \text{auth})$

- Verify:
 - P_0, P_1 registered
 - Valid authorization
- Create transaction:
 - Fresh tid
 - Status pending

Guarantee:

- No unauthorized transactions

If valid and sufficient balance:

$$\text{bal}(P_0) \leftarrow \text{bal}(P_0) - m$$

$$\text{bal}(P_1) \leftarrow \text{bal}(P_1) + m$$

- Status $\rightarrow$ committed
- Generate success evidence $e_{tx}^\vee$
- Notify P_0, P_1 (possibly delayed)

If checks fail or cancellation triggered:

- No net balance change
- Status $\rightarrow$ aborted
- Generate failure evidence e_{tx}^x
- Notify parties

Key property:

- No residual debit after failure

Adversarial Network Model

- Adversary controls:
 - Message delays
 - Ordering
- But:
 - Final status eventually delivered

Implication:

- Temporary uncertainty allowed
- Eventual consistency guaranteed

Status and Proof Interfaces

Status query:

$(\text{GetStatus}, U, \text{tid})$

Success proof:

$(\text{GetSuccessProof}, U, \text{tid})$

Failure proof:

$(\text{GetFailureProof}, U, \text{tid})$

Purpose:

- Enable provability and dispute resolution **without delay**.

Leakage Function $\mathcal{L}$ (Adversary View)

Principle:

- $\mathcal{F}_{\text{pay}}$ reveals only minimal information to adversary $\mathcal{A}$
- Captures unavoidable observables in real systems

On transaction initiation:

$$\mathcal{L}_{\text{init}} = (\text{start}, \text{tid}, P_0, P_1, m)$$

- Identities may be:
 - real identities (baseline model), or
 - pseudonyms (privacy-enhanced variant)
- Amount m is typically unavoidable (bank settlement)

On transaction completion:

$$\mathcal{L}_{\text{done}} = (\text{done}, \text{tid}, \text{status})$$

- $\text{status} \in \{\text{committed}, \text{aborted}\}$

Timing leakage:

$$\mathcal{L}_{\text{time}} = (\text{delay pattern})$$

- Adversary observes message timing and ordering
- Models network-level side channels

No additional leakage:

- No bank account details
- No authentication secrets
- No full transaction history (unless linked explicitly)

Transaction leakages

Two design variants:

- **Baseline (UPI-like):**

$$\mathcal{L} \ni (P_0, P_1, m) \quad (\text{identity leakage})$$

- **Privacy-enhanced:**

$$\mathcal{L} \ni (\tilde{P}_0, \tilde{P}_1, m) \quad (\text{pseudonyms})$$

- Fresh pseudonyms per transaction
- Prevents linkability

Interpretation:

- Leakage defines the *privacy boundary* of the system
- Stronger privacy $\Rightarrow$ smaller $\mathcal{L}$

File dispute:

$(\text{FileDispute}, U, \text{tid})$

Resolution by regulator:

$(\text{ResolveDispute}, R, \text{tid})$

- Uses internal logs $E(tx)$
- Can:
 - Verify outcome
 - Reveal accountable identities
 - Enforce remedy

- **No unauthorized transactions**
- **Bank safety:**
 - No credit without debit
 - No double spending
- **Consistency:**
 - Parties converge to same final state
- **Provability:**
 - Success and failure are verifiable

Privacy Guarantees

- Minimal disclosure:
 - Payer sees only required payee info
 - Payee sees limited payer info
- Banks see only settlement data
- Regulator sees full data only on dispute

Goal:

- No unnecessary identity leakage
- No transaction linkability

Dispute resolution guarantees: Two Variants

In-person: $\mathcal{F}_{\text{pay}}^{\text{local}}$

- Strong guarantees:
 - Failure finality
 - Mutual provability

Remote: $\mathcal{F}_{\text{pay}}^{\text{remote}}$

- Weaker guarantees:
 - External dispute resolution
 - Partial failure guarantees

Key Strengths of UPI

- Excellent usability:
 - No hardware
 - Instant settlement
 - Direct bank transfer
- Massive adoption and interoperability

Key Weakness: Centralised Trust

- NPCI is a **single trusted intermediary**
- Responsible for:
 - Correctness
 - Privacy
 - Dispute resolution
- Under adversarial model:
 - Most guarantees fail

Digital cash (David Chaum, 1984)

Goal:

- Enable payments with:
 - **Strong privacy (anonymity)**
 - **Bank-backed value**

Key mechanism: Blind signatures

- User prepares a random coin C
- Blinds it: $\tilde{C} = \text{Blind}(C)$ ([Click here for blind signatures](#))
- Bank signs $\tilde{C}$ without seeing C but verifying and debiting amount
- User unblinds to obtain:

$$\sigma = \text{Sign}(C)$$

Payment:

- Payer sends (C, σ) to payee
- verifies bank signature
- Payee deposits coin with bank

Digital cash

Properties:

- **Unlinkability:** Bank cannot link withdrawal to spend
- **Correctness:** Only bank-signed coins are valid

Shortcomings: poor dispute resolution

- **Double-spending issue:** Same coin can be copied and reused. Requires online check or post-facto detection
- In the offline version, when the Payee is not connected to the internet, the Payee has no way of knowing if the Payer uses an already used coin (double spending). The bank will find out when the Payee tries to deposit the coin, but who reimburses the Payee? That the Payer will eventually get caught is the only deterrence against double-spending.
- In an alternative online version, the Payee is required to check with the Bank that the coin is unspent before completing the transaction. A delayed response from the Bank can cause a denial of service, though no loss of money.

A good solution that overcomes these will be most welcome.

Open Questions

- Deployment feasibility of cryptographic protocols
- Regulatory compatibility
- Balancing auditability vs privacy
- Extending to online (non co-present) settings